2026 春

程序设计实习
第三讲

运算符重载

杨　帅
williamyang@pku.edu.cn

上节回顾

- 构造函数 & 复制构造函数 & 析构函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

目　录

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

目　录

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

主要内容

- 两种运算符重载的实现方式
- 常见的运算符重载
  - 流运算符：>>，<<
  - 自增运算符++，自减运算符--

自定义数据类型与运算符重载

- C++ 预定义了一组运算符，用来表示对数据的运算
  - +, -, *, /, %, ^, &, ~, !, |, =, <<, >>, !=, ...
  - **只能用于<u>基本</u>的数据类型**
    - 整型，实型，字符型，逻辑型，...

运算符重载

- C++提供了数据抽象的手段
  - 允许用户自己定义数据类型：类
  - 通过调用类的成员函数，对它的对象进行操作
- 有时 用类的成员函数来操作对象时，很不方便
- 例如：
  - 在数学上，两个复数可以直接进行 + / - 等运算
  - 但在C++中，直接将 + 或 - 用于复数是不允许的

运算符重载

- 希望：对一些抽象数据类型（即自定义数据类型），也能够直接使用C++提供的运算符
  - 程序更简洁
  - 代码更容易理解
- 例如：
  - complex_a 和 complex_b 是两个复数对象；
  - 求两个复数的和， 希望能直接写：

```
complex_a + complex_b
```

运算符重载

- **运算符重载**
  - 对已有的运算符(C++中预定义的运算符)**赋予多重**的含义
  - 使**同一运算符**作用于**不同类型的数据**时导致不同类型的行为
- 目的
  - 扩展C++中提供的运算符的适用范围，以用于类所表示的抽象数据类型
- 同一个运算符，对不同类型的操作数，所发生的行为不同
  - `(5, 10i) + (4, 8i) = (9, 18i)`
  - `5 + 4 = 9`

运算符重载

- 运算符重载的实质是函数重载

```
返回值类型 operator 运算符 ( 形参表 )
{
        ...
}
```

运算符重载

- 在程序编译时：
  - 把 **运算符的表达式** → **运算符函数的调用**
  - 把 **运算符的操作数** → **运算符函数的参数**
  - 运算符被**多次重载**时，根据 实参的类型 决定调用哪个运算符函数
  - 运算符可以被重载成**普通函数**
  - 也可以被重载成**类的成员函数**

运算符重载示例

重载为成员函数时，参数个数为运算符目数减 1

重载为普通函数时，参数个数为运算符目数

```
class Complex {
public:
    double real,  imag;
    Complex( double r = 0.0,  double i= 0.0 ):real(r),  imag(i)   {     }
    Complex operator-(const Complex & c);
};
Complex operator+(const Complex & a,  const Complex & b){
    return Complex(a.real+b.real,  a.imag+b.imag);
    //返回一个临时对象
}
Complex Complex::operator-(const Complex & c){
    return Complex(real - c.real,  imag - c.imag);
    //返回一个临时对象
}
```

运算符重载示例


```
int main(){
    Complex a(4, 4),  b(1, 1),  c;
    c = a + b;     //等价于?
    cout << c.real << ", " << c.imag << endl;
    cout << (a-b).real << ", " << (a-b).imag << endl;
    //a-b等价于?
    return 0;
}
```

```
5, 5
3, 3
```

运算符重载示例


```
int main(){
    Complex a(4, 4),  b(1, 1),  c;
    c = a + b;     //等价于operator+(a, b)
    cout << c.real << ", " << c.imag << endl;
    cout << (a-b).real << ", " << (a-b).imag << endl;
    //a-b等价于a.operator-(b)
    return 0;
}
```

```
5, 5
3, 3
```

a + b 等价于 operator+(a, b);
a - b 等价于 a.operator-(b);

测一测

- 如果将[]运算符重载成一个类的成员函数，则该重载函数有几个参数?

A. 0

B. 1

C. 2

D. 3

测一测

- 如果将[]运算符重载成一个类的成员函数，则该重载函数有几个参数?

A. 0

B. 1

C. 2

D. 3

目　录

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

赋值运算符 '=' 重载

- 赋值运算符 两边的类型 可以 **不匹配**
  - 把一个 int 类型变量 赋值给一个 Complex 对象
  - 把一个 char * 类型的字符串 赋值给一个字符串对象
  - 需要 **重载赋值运算符 '='**
- **赋值运算符 '=' 只能重载为 成员函数**

赋值运算符 '=' 重载

- 编写一个**长度可变的字符串类String**
  - 包含一个**char * 类型**的成员变量

    → 指向动态分配的存储空间
  - 该存储空间用于存放 '\0' 结尾的字符串

```
class String {
    char * str;
public:
    String () : str(new char[1]) { str[0] = 0; }
    const char * c_str() { return str; }
    String & operator = (const char * s);
    ~String() { delete [] str; }
};
```

赋值运算符 '=' 重载


```
//重载 "=" → obj = "hello" 能够成立
String & String::operator = (const char * s){
    delete [] str;
    str = new char[strlen(s)+1];
    strcpy(str, s);
    return *this;
}

int main(){
    String s;
    s = "Good Luck," ;    //等价于 s.operator = ("Good Luck,");
    cout << s.c_str() << endl;
    // String s2 = "hello!"; //不注释掉就会出错，没定义构造函数
    s = "Shenzhou 13!"; //等价于 s.operator = ("Shenzhou 13!");
    cout << s.c_str() << endl;
    return 0;
}
```

```
Good Luck,
Shenzhou 13!
```

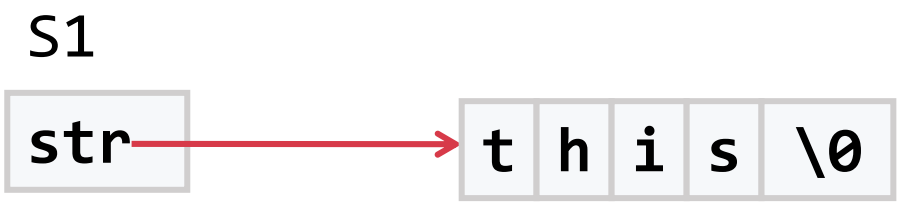
重载赋值运算符的意义 – 浅复制和深复制

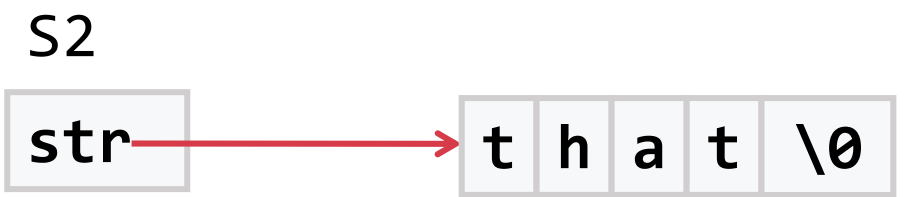
- **S1 = S2;**
  - **浅复制/浅拷贝**
  - 执行逐个字节的复制工作

```
String S1, S2;
S1 = "this";
S2 = "that";
S1 = S2;
```

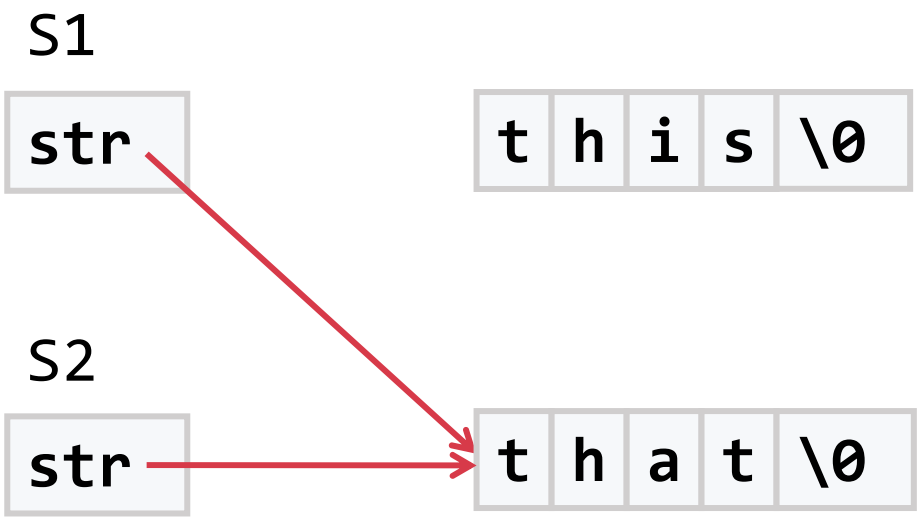
浅复制


```
String S1, S2;
S1 = "this";
S2 = "that";
```

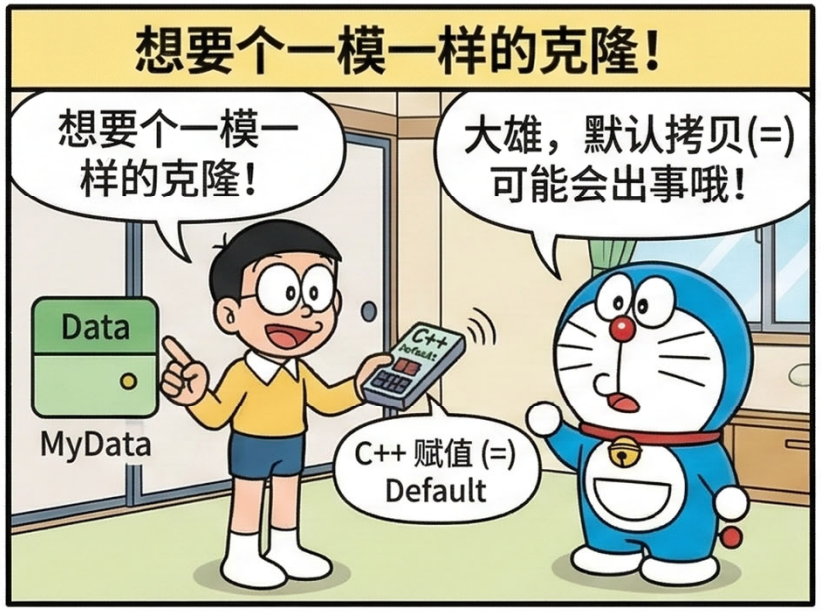


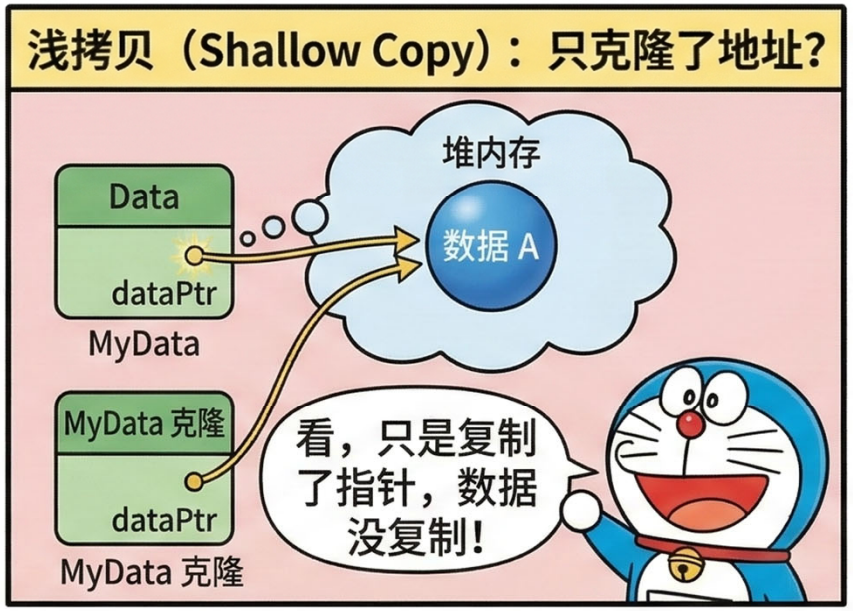


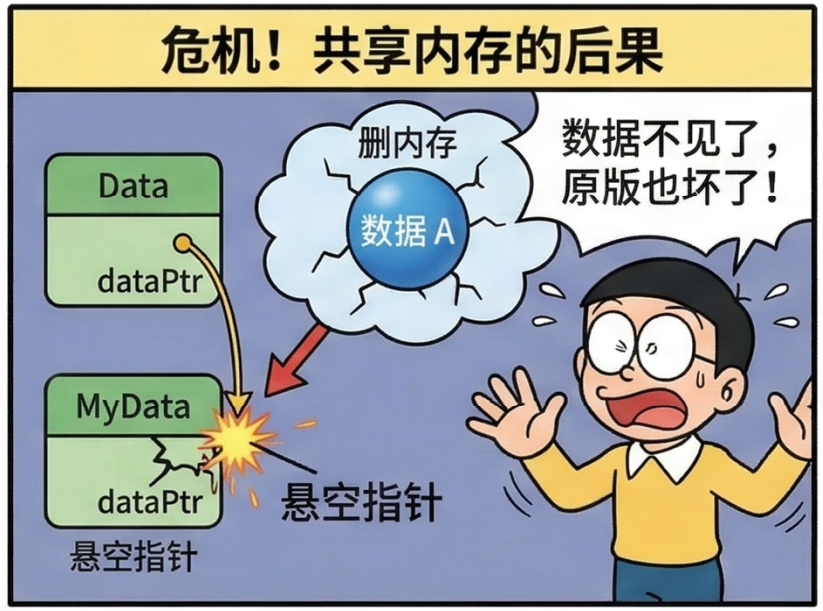

```
S1 = S2;
```



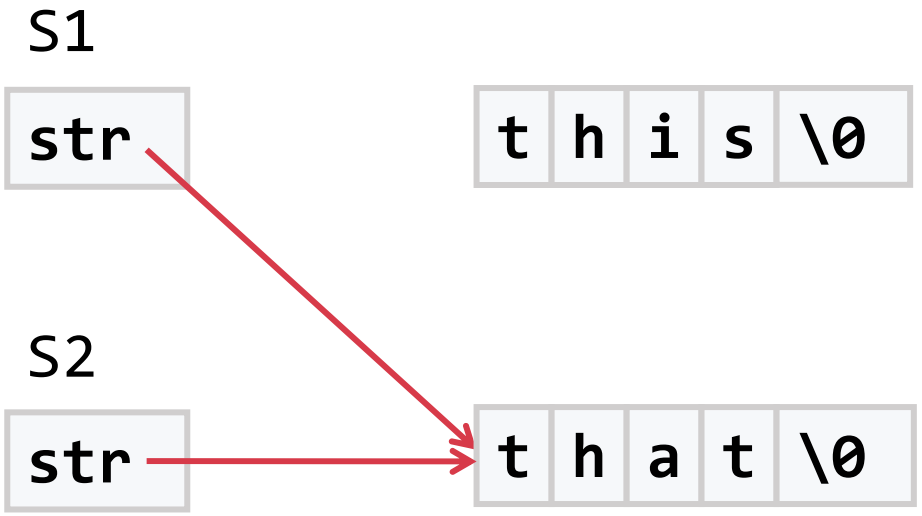
浅复制


想要个一模一样的克隆！
想要个一模一样的克隆！
大雄，默认拷贝(=)可能会出事哦！
Data
MyData
C++ 赋值 (=) Default


浅拷贝（Shallow Copy）：只克隆了地址？
堆内存
数据 A
Data
dataPtr
MyData
MyData 克隆
dataPtr
MyData 克隆
看，只是复制了指针，数据没复制！


危机！共享内存的后果
删内存
数据 A
数据不见了，原版也坏了！
Data
dataPtr
MyData
dataPtr
悬空指针
悬空指针

```
S1 = S2;
```


S1
str
t h i s \0
S2
str
t h a t \0

浅拷贝的问题

- 如不定义自己的赋值运算符，那么S1=S2实际上导致**S1.str和S2.str指向同一地方**
- 如果S1对象消亡，析构函数将释放S1.str指向的空间，则S2消亡时还要释放一次，不妥
- 另外，如果执行 S1 = "other"；会导致S2.str指向的地方被 delete

重载赋值运算符的意义 – 浅复制和深复制

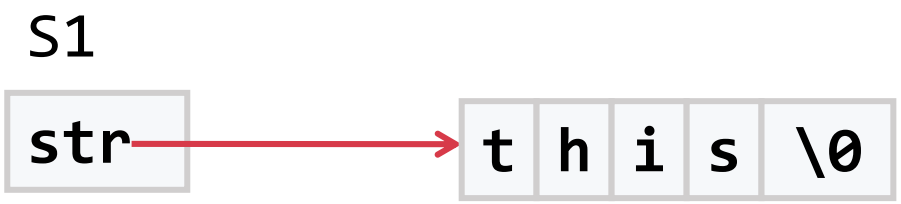
- **深复制/深拷贝**
  - 将一个**对象中指针变量指向的内容**

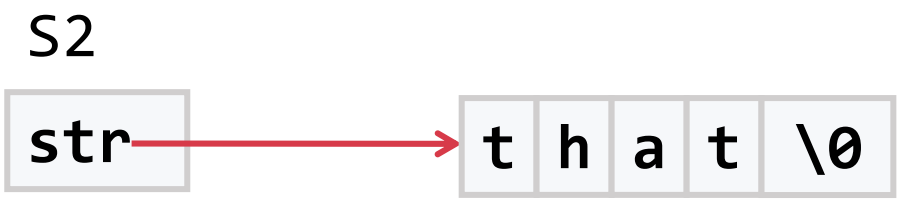
    → 复制到另一个对象中指针成员对象指向的地方

```
String S1, S2;
S1 = "this";
S2 = "that";
S1 = S2;
```

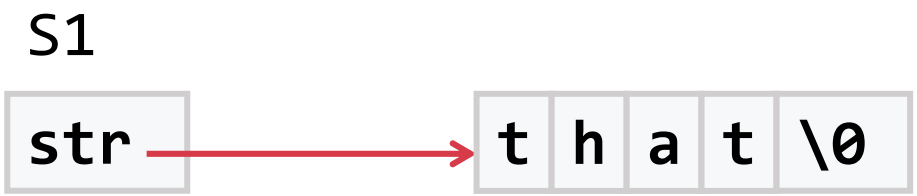
深复制

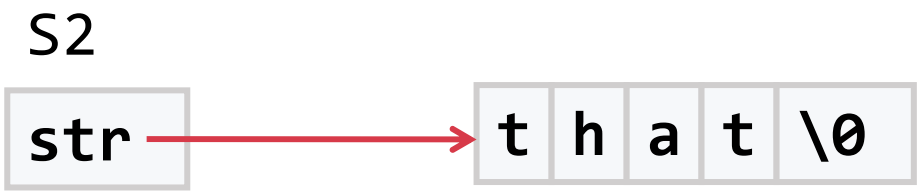

```
String S1, S2;
S1 = "this";
S2 = "that";
```






```
S1 = S2;
```





深复制

- 在 class String 里添加成员函数:

```
String & operator = (const String & s) {
    delete [] str;
    str = new char[strlen(s.str)+1];
    strcpy(str, s.str);
    return * this;
}
```

- 这么做就够了吗？还有什么需要改进的地方？

思　考

- 考虑下面语句：

```
String & operator = (const String & s) {
    delete [] str;
    str = new char[strlen(s.str)+1];
    strcpy(str, s.str);
    return * this;
}
```

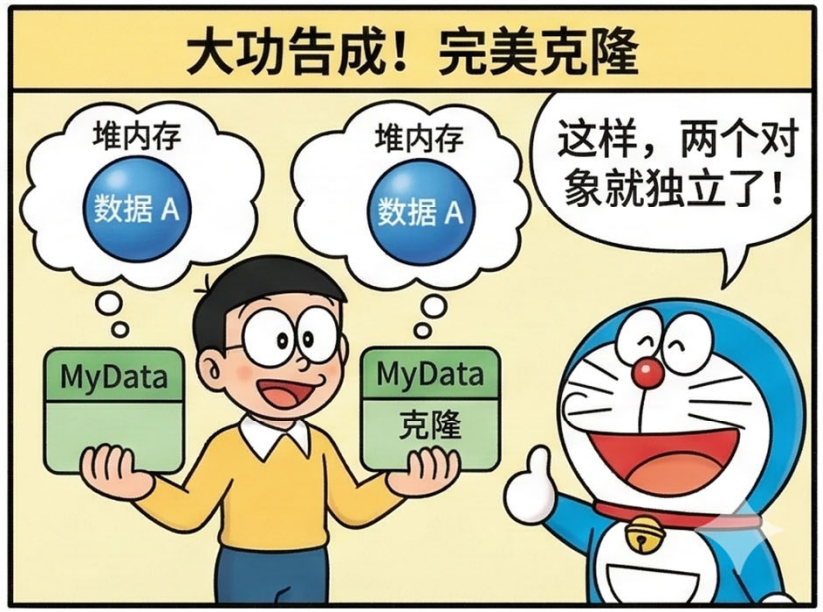
```
String s;
s = "Hello";
s = s;
```

- 是否会有问题？

正确写法：

```
String & operator = (const String & s){
    if( this == &s ) return * this;
    delete [] str;
    str = new char[strlen(s.str)+1];
    strcpy( str, s.str);
    return * this;
}
```

```
String & operator = (const String & s){
    if( this == &s ) return * this;
    delete [] str;
    str = new char[strlen(s.str)+1];
    strcpy( str, s.str);
    return * this;
}
```



对 operator = 返回值类型的讨论

void 好不好?

String 好不好?

为什么是 String &

- 运算符重载时，好的风格 – 尽量保留运算符原本的特性
- 考虑：a = b = c; 和 (a=b)=c; //(a=b)表达式是a的引用
- 分别等价于：

```
a.operator=(b.operator=(c));
(a.operator=(b)).operator=(c);
```

思　考

- 上面的String类是否就没有问题了
- 为String类编写 **复制构造函数** 时

  会面临和 '=' 同样的问题，用同样的方法处理

```
String(const String & s) {
    str = new char[strlen(s.str)+1];
    strcpy(str, s.str);
}
```

目　录

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

运算符重载为友元

- 一般情况下，推荐将运算符重载为**类的成员函数**
- 重载为成员函数不能满足使用要求时，重载为**普通函数**
- 又不能访问类的私有成员，所以需要将运算符重载为**友元**

```
class Complex {
public:
    Complex(double r= 0.0,  double i= 0.0):real(r), imaginary(i){
    }; //constructor
    Complex operator+(int r){
        return Complex(real + r,  imaginary);
    }
private:
    double real;        // real part
    double imaginary;   // imaginary part
};
```

运算符重载为友元

- 经过上述重载后:

```
Complex c ;
c = c + 5;  //有定义，相当于 c = c.operator +(5);
但是:
c = 5 + c; //编译出错
```

- 为了使得上述表达式能成立，需要将 + 重载为普通函数

```
Complex operator + (int n,  const Complex & c) {
    return Complex(c.real + n, c.imaginary);
}
```

运算符重载为友元

- 但是普通函数不能访问私有成员，所以需要将运算符+重载为友元

```
class Complex {
public:
    Complex(double r= 0.0,  double i= 0.0):
        real(r),  imaginary(i){ }; //constructor
    Complex operator+(int r){
        return Complex(real + r,  imaginary);
    }
    friend Complex operator+(int r,  const Complex & C);
private:
    double real;              // real part
    double imaginary;  // imaginary part
};
```

目　录

- 运算符重载基本概念
- 赋值运算符的重载
- 运算符重载为友元函数
- 流插入运算符和流提取运算符的重载
- 类型转换运算符的重载
- 自增、自减运算符的重载

流插入运算符的重载

- `cout << 5 << "this";`

  为什么能够成立?

- cout是什么?

  "<<" 为什么能用在 cout 上?

流插入运算符的重载

- cout 是在 iostream 中定义的，**ostream 类的对象**

  "<<" 能用在 cout 上

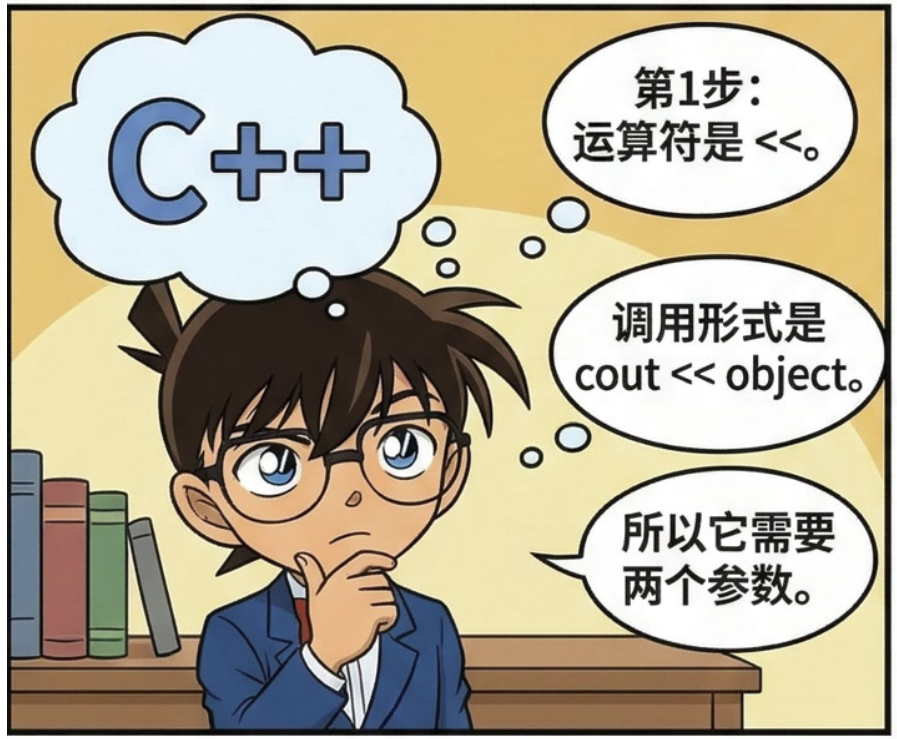
  因为在iostream里对 "<<" 进行了重载

- 考虑怎么重载才能使得

  cout << 5;

  和 **cout << "this";**

  都能成立?



流插入运算符的重载

- 有可能按以下方式重载：

```
void operator<<( ostream & o,  int n ){
    Output(n);
}
```

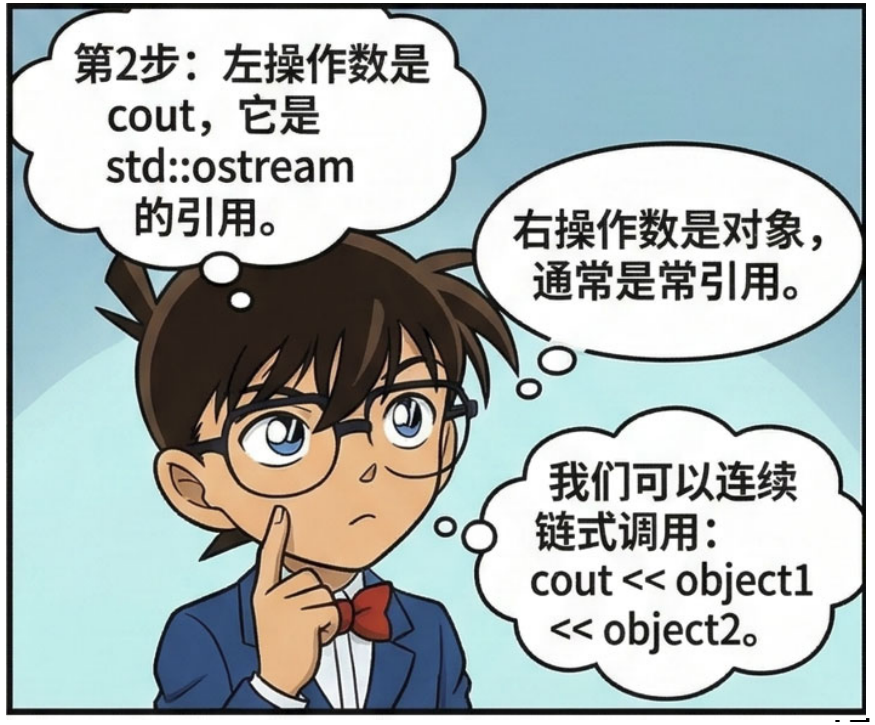
- 假定Output( )是一个能将整数n输出到屏幕上的函数，至于其内部怎么实现，不必深究

```
void operator<<( ostream & o,  const char * s ){
    Output(s);
}
```

流插入运算符的重载

- `cout << 5;` 即 `operator<<(cout, 5);`
- `cout << "this";` 即 `operator<<(cout, "this");`

- 怎么重载才能使得
- `cout << 5 << "this";`
- 成立？



流插入运算符的重载


```
ostream & operator<<( ostream & o,  int n ){
    Output(n);
    return o;
}
```

流插入运算符的重载


```
ostream & operator<<( ostream & o,  int n ){
    Output(n);
    return o;
}
```

- 假定Output ( )是一个能将整数n输出到屏幕上的函数，至于其内部怎么实现，不必深究

```
ostream & operator<<( ostream & o,  const char * s ){
    Output(s);
    return o;
}  //用引用作为返回值是为了提高效率
```

- 当然，也可能是ostream类将 << 重载为成员函数

流插入运算符的重载

- `cout << 5 << "this";`
- 本质上的函数调用的形式是什么?
- `operator<<(operator<<(cout, 5) , "this");`

流插入运算符的重载

- 假定下面程序输出为 5hello，请问该补写些什么？

```
class CStudent{
public:
    int nAge;
};
int main(){
    CStudent s ;
    s.nAge = 5;
    cout << s << "hello";
    return 0;
}
```

流插入运算符的重载

- `cout << s << "hello";`

```
ostream & operator<< ( ostream & o, const CStudent & s ){
    o << s.nAge;
    return o;
}
```

流插入运算符的重载

- 事实上在 iostream 里是将 << 重载成 **成员函数**

```
class ostream{
    ostream & operator<< (int n) {
        Output(n);
        return * this;
    }
};
```

- 那么 **cout << n++ << n;** 的函数调用形式是什么呢?

流插入运算符的重载

- **cout << n++ << n;**

```
cout.operator << (n++).operator << (n);
```

- 实际上，上面这条语句可以直接写在程序里，其效果和

  cout << n++ << n;

  完全一样